05 - EVM 실행 시스템

개요

이더리움 가상 머신(EVM)은 Geth의 핵심 컴포넌트로, 스마트 컨트랙트의 바이트코드를 실행하고 트랜잭션을 처리하는 역할을 담당합니다. EVM은 스택 기반의 가상 머신으로, 256비트 워드를 사용하며 가스 기반의 실행 모델을 통해 계산 비용을 관리합니다.

EVM 시스템은 다음과 같은 주요 컴포넌트들로 구성됩니다:

- **EVM 인스턴스**: 실행 환경과 컨텍스트 관리
- **인터프리터**: 바이트코드 해석 및 실행
- **컨트랙트**: 실행 단위와 코드 관리
- **메모리 및 스택**: 실행 중 데이터 저장
- **가스 계산**: 실행 비용 관리

핵심 구조체 및 인터페이스

EVM 구조체

`vm.EVM` 은 이더리움 가상 머신의 메인 구조체로, 실행에 필요한 모든 컨텍스트와 상태를 관리합니다:

```
type EVM struct {
// 블록체인 컨텍스트 정보
Context BlockContext
TxContext
// 상태 데이터베이스 접근
```

```
StateDB StateDB

// 옵코드 점프 테이블

table *JumpTable

// 호출 스택 깊이

depth  int

// 체인 설정 및 규칙

chainConfig *params.ChainConfig

chainRules params.Rules

// EVM 설정

Config Config

// 실행 제어

abort atomic.Bool

callGasTemp  uint64

// 프리컴파일된 컨트랙트

precompiles  map[common.Address]PrecompiledContract

// 점프 목적지 캐시

jumpDests JumpDestCache

// 해시 계산기

hasher crypto.KeccakState

hasherBuf common.Hash

// 실행 상태

readOnly  bool

returnData []byte
```

```
}
```

블록 및 트랜잭션 컨텍스트

EVM 실행에 필요한 블록체인 정보를 제공하는 컨텍스트 구조체들:

```
type BlockContext struct {
CanTransfer CanTransferFunc // 잔액 확인 함수
Transfer TransferFunc // 이더 전송 함수
GetHash GetHashFunc // 블록 해시 조회 함수
// 블록 정보
Coinbase common.Address // 코인베이스 주소
GasLimit  uint64  // 가스 한도
BlockNumber *big.Int // 블록 번호
Time  uint64  // 타임스탬프
Difficulty *big.Int // 난이도
BaseFee *big.Int // 기본 수수료
BlobBaseFee *big.Int // 블롭 기본 수수료
Random *common.Hash // 랜덤 값
}


type TxContext struct {
Origin common.Address // 트랜잭션 발신자
GasPrice *big.Int // 가스 가격
```

```
BlobHashes []common.Hash // 블롭 해시들

BlobFeeCap *big.Int // 블롭 수수료 상한

AccessEvents *state.AccessEvents // 상태 접근 이벤트

}
```

주요 프로세스

EVM 인스턴스 생성

`NewEVM()` 함수는 새로운 EVM 인스턴스를 생성하고 초기화합니다:

```
func  NewEVM(blockCtx BlockContext, statedb StateDB, chainConfig *params.ChainCo

evm := &EVM{

Context: blockCtx,

StateDB: statedb,

Config: config,

chainConfig: chainConfig,

chainRules: chainConfig.Rules(blockCtx.BlockNumber, blockCtx.Random != nil, bloc

jumpDests: newMapJumpDests(),

hasher: crypto.NewKeccakState(),

}

// 프리컴파일된 컨트랙트 설정

evm.precompiles = activePrecompiledContracts(evm.chainRules)

// 체인 규칙에 따른 명령어 세트 선택
```

```
switch {
case evm.chainRules.IsOsaka:
evm.table = &osakaInstructionSet
case evm.chainRules.IsPrague:
evm.table = &pragueInstructionSet
case evm.chainRules.IsCancun:
evm.table = &cancunInstructionSet
// ... 기타 하드포크별 명령어 세트
default:
evm.table = &frontierInstructionSet
}
return evm
}
```

컨트랙트 호출 (Call)

`EVM.Call()` 메서드는 기존 컨트랙트를 호출하여 실행합니다:

```
func (evm *EVM) Call(caller common.Address, addr common.Address, input []byte, g
// 트레이싱 시작
if evm.Config.Tracer != nil {
evm.captureBegin(evm.depth, CALL, caller, addr, input, gas, value.ToBig())
defer func(startGas uint64) {
```

```
evm.captureEnd(evm.depth, startGas, leftOverGas, ret, err)
}(gas)
}
// 호출 깊이 제한 확인
if  evm.depth > int(params.CallCreateDepth) {
return  nil, gas, ErrDepth
}
// 잔액 확인
if !value.IsZero() && !evm.Context.CanTransfer(evm.StateDB, caller, value) {
return  nil, gas, ErrInsufficientBalance
}
snapshot := evm.StateDB.Snapshot()
p, isPrecompile := evm.precompile(addr)
// 계정 존재 확인 및 생성
if !evm.StateDB.Exist(addr) {
if !isPrecompile && evm.chainRules.IsEIP158 && value.IsZero() {
return  nil, gas, nil
}
evm.StateDB.CreateAccount(addr)
}
// 이더 전송
evm.Context.Transfer(evm.StateDB, caller, addr, value)
if  isPrecompile {
```

```
// 프리컴파일된 컨트랙트 실행
ret, gas, err = RunPrecompiledContract(p, input, gas, evm.Config.Tracer)
} else {
// 일반 컨트랙트 실행
code := evm.resolveCode(addr)
if len(code) == 0 {
ret, err = nil, nil
} else {
contract := NewContract(caller, addr, value, gas, evm.jumpDests)
contract.SetCallCode(evm.resolveCodeHash(addr), code)
ret, err = evm.Run(contract, input, false)
gas = contract.Gas
}
}
// 에러 처리 및 상태 롤백
if err != nil {
evm.StateDB.RevertToSnapshot(snapshot)
if err != ErrExecutionReverted {
gas = 0
}
}
return ret, gas, err
```

```
}
```

컨트랙트 생성 (Create)

`EVM.Create()` 메서드는 새로운 컨트랙트를 배포합니다:

```
func (evm *EVM) Create(caller common.Address, code []byte, gas  uint64, value *u
// 컨트랙트 주소 계산
contractAddr = crypto.CreateAddress(caller, evm.StateDB.GetNonce(caller))
return  evm.create(caller, code, gas, value, contractAddr, CREATE)
}

func (evm *EVM) create(caller common.Address, code []byte, gas  uint64, value *u
// 깊이 및 잔액 확인
if  evm.depth > int(params.CallCreateDepth) {
return  nil, common.Address{}, gas, ErrDepth
}
if !evm.Context.CanTransfer(evm.StateDB, caller, value) {
return  nil, common.Address{}, gas, ErrInsufficientBalance
}
// 논스 증가
nonce := evm.StateDB.GetNonce(caller)
evm.StateDB.SetNonce(caller, nonce+1, tracing.NonceChangeContractCreator)
// 주소 충돌 확인
```

```
contractHash := evm.StateDB.GetCodeHash(address)
storageRoot := evm.StateDB.GetStorageRoot(address)
if evm.StateDB.GetNonce(address) != 0 ||
(contractHash != (common.Hash{}) && contractHash != types.EmptyCodeHash) ||
(storageRoot != (common.Hash{}) && storageRoot != types.EmptyRootHash) {
return nil, common.Address{}, 0, ErrContractAddressCollision
}
snapshot := evm.StateDB.Snapshot()
// 계정 생성
if !evm.StateDB.Exist(address) {
evm.StateDB.CreateAccount(address)
}
evm.StateDB.CreateContract(address)
if evm.chainRules.IsEIP158 {
evm.StateDB.SetNonce(address, 1, tracing.NonceChangeNewContract)
}
// 이더 전송
evm.Context.Transfer(evm.StateDB, caller, address, value)
// 컨트랙트 초기화
contract := NewContract(caller, address, value, gas, evm.jumpDests)
contract.SetCallCode(common.Hash{}, code)
contract.IsDeployment = true
ret, err = evm.initNewContract(contract, address)
```

```
if err != nil && (evm.chainRules.IsHomestead || err != ErrCodeStoreOutOfGas) {
evm.StateDB.RevertToSnapshot(snapshot)
if err != ErrExecutionReverted {
contract.UseGas(contract.Gas, evm.Config.Tracer, tracing.GasChangeCallFailedExec
}
}
return ret, address, contract.Gas, err
}
```

인터프리터와 바이트코드 실행

인터프리터 구조

`vm.Interpreter` 는 바이트코드를 해석하고 실행하는 핵심 컴포넌트입니다:

```
type ScopeContext struct {
Memory *Memory // 메모리 관리
Stack *Stack // 스택 관리
Contract *Contract // 실행 중인 컨트랙트
}
```

바이트코드 실행 루프

`EVM.Run()` 메서드는 컨트랙트의 바이트코드를 실행합니다:

```go
func (evm *EVM) Run(contract *Contract, input []byte, readOnly  bool) (ret []byt

// 호출 깊이 증가

evm.depth++

defer  func() { evm.depth-- }()

// 읽기 전용 모드 설정

if  readOnly && !evm.readOnly {

evm.readOnly = true

defer  func() { evm.readOnly = false }()

}

// 코드가 없으면 즉시 반환

if  len(contract.Code) == 0 {

return  nil, nil

}

var (

op OpCode

jumpTable = evm.table

mem = NewMemory()

stack = newstack()

callContext = &ScopeContext{

Memory: mem,

Stack: stack,

Contract: contract,
```

```
}
pc = uint64(0) // 프로그램 카운터
cost  uint64  // 가스 비용
)
defer  func() {
returnStack(stack)
mem.Free()
}()
contract.Input = input
// 메인 실행 루프
for {
// 옵코드 가져오기
op = contract.GetOp(pc)
operation := jumpTable[op]
cost = operation.constantGas
// 스택 검증
if  sLen := stack.len();  sLen < operation.minStack {
return  nil, &ErrStackUnderflow{stackLen: sLen, required: operation.minStack}
} else  if  sLen > operation.maxStack {
return  nil, &ErrStackOverflow{stackLen: sLen, limit: operation.maxStack}
}
// 기본 가스 소모
if  contract.Gas < cost {
```

```
return nil, ErrOutOfGas
}
contract.Gas -= cost
// 동적 가스 계산
var memorySize uint64
if operation.dynamicGas != nil {
if operation.memorySize != nil {
memSize, overflow := operation.memorySize(stack)
if overflow {
return nil, ErrGasUintOverflow
}
memorySize, overflow = math.SafeMul(toWordSize(memSize), 32)
if overflow {
return nil, ErrGasUintOverflow
}
}
dynamicCost, err := operation.dynamicGas(evm, contract, stack, mem, memorySize)
cost += dynamicCost
if err != nil {
return nil, fmt.Errorf("%w: %v", ErrOutOfGas, err)
}
if contract.Gas < dynamicCost {
return nil, ErrOutOfGas
```

```
}
contract.Gas -= dynamicCost
}
// 메모리 확장
if memorySize > 0 {
mem.Resize(memorySize)
}
// 옵코드 실행
res, err := operation.execute(&pc, evm, callContext)
if err != nil {
break
}
pc++
}
if err == errStopToken {
err = nil
}
return res, err
}
```

가스 계산 메커니즘

가스 상수

EVM에서 사용되는 기본 가스 비용들:

```
const (
GasQuickStep uint64 = 2 // 빠른 연산
GasFastestStep uint64 = 3 // 가장 빠른 연산
GasFastishStep uint64 = 4 // 비교적 빠른 연산
GasFastStep uint64 = 5 // 빠른 연산
GasMidStep uint64 = 8 // 중간 연산
GasSlowStep uint64 = 10 // 느린 연산
GasExtStep uint64 = 20 // 확장 연산
)
```

호출 가스 계산

`callGas()` 함수는 컨트랙트 호출 시 사용할 가스를 계산합니다:

```
func callGas(isEip150 bool, availableGas, base uint64, callCost *uint256.Int)
if isEip150 {
// EIP-150: 63/64 규칙 적용
availableGas = availableGas - base
gas := availableGas - availableGas/64
if !callCost.IsUint64() || gas < callCost.Uint64() {
return gas, nil
}
}
```

```
if !callCost.IsUint64() {
return  0, ErrGasUintOverflow
}
return  callCost.Uint64(), nil
}
```

동적 가스 계산

각 옵코드는 실행 시점에 동적으로 가스를 계산할 수 있습니다:

```
type operation struct {
execute executionFunc // 실행 함수
constantGas  uint64  // 고정 가스 비용
dynamicGas gasFunc // 동적 가스 계산 함수
minStack  int  // 최소 스택 크기
maxStack  int  // 최대 스택 크기
memorySize memorySizeFunc // 메모리 크기 계산 함수
}
```

메모리 및 스택 관리

메모리 구조

`vm.Memory` 는 EVM 실행 중 임시 데이터를 저장합니다:

```go
type Memory struct {
store []byte  // 메모리 저장소
lastGasCost  uint64  // 마지막 가스 비용
}

func (m *Memory) Set(offset, size  uint64, value []byte) {
if  size > 0 {
if  offset+size > uint64(len(m.store)) {
panic("invalid memory: store empty")
}
copy(m.store[offset:offset+size], value)
}
}

func (m *Memory) Set32(offset  uint64, val *uint256.Int) {
if  offset+32 > uint64(len(m.store)) {
panic("invalid memory: store empty")
}
val.PutUint256(m.store[offset:])
}

func (m *Memory) Resize(size  uint64) {
```

```go
if  uint64(m.Len()) < size {
m.store = append(m.store, make([]byte, size-uint64(m.Len()))...)
}
}
```

스택 구조

`vm.Stack` 은 256비트 정수를 저장하는 스택입니다:

```go
type Stack struct {
data []uint256.Int
}

func (st *Stack) push(d *uint256.Int) {
st.data = append(st.data, *d)
}

func (st *Stack) pop() (ret uint256.Int) {
ret = st.data[len(st.data)-1]
st.data = st.data[:len(st.data)-1]
return
}

func (st *Stack) swap1() {
```

```
st.data[st.len()-2], st.data[st.len()-1] = st.data[st.len()-1], st.data[st.len()
}
```

컨트랙트 관리

컨트랙트 구조

vm.Contract 는 실행 중인 컨트랙트의 상태를 관리합니다:

```
type Contract struct {
caller common.Address // 호출자 주소
address common.Address // 컨트랙트 주소
jumpDests JumpDestCache // 점프 목적지 캐시
analysis BitVec // 코드 분석 결과
Code []byte  // 바이트코드
CodeHash common.Hash // 코드 해시
Input []byte  // 입력 데이터
IsDeployment  bool  // 배포 여부
IsSystemCall  bool  // 시스템 호출 여부
Gas  uint64  // 남은 가스
value *uint256.Int // 전송 값
}
```

```
func  NewContract(caller common.Address, address common.Address, value *uint256.
```

```
if  jumpDests == nil {
jumpDests = newMapJumpDests()
}
return &Contract{
caller: caller,
address: address,
jumpDests: jumpDests,
Gas: gas,
value: value,
}
}
```

점프 목적지 검증

컨트랙트는 JUMP 명령어의 유효성을 검증합니다:

```
func (c *Contract) validJumpdest(dest *uint256.Int) bool {
udest, overflow := dest.Uint64WithOverflow()
if  overflow || udest >= uint64(len(c.Code)) {
return false
}
// JUMPDEST 옵코드인지 확인
if  OpCode(c.Code[udest]) != JUMPDEST {
return false
```

```
}
return  c.isCode(udest)
}


func (c *Contract) isCode(udest  uint64) bool {
if  c.analysis != nil {
return  c.analysis.codeSegment(udest)
}
if  c.CodeHash != (common.Hash{}) {
analysis, exist := c.jumpDests.Load(c.CodeHash)
if !exist {
analysis = codeBitmap(c.Code)
c.jumpDests.Store(c.CodeHash, analysis)
}
c.analysis = analysis
return  analysis.codeSegment(udest)
}
// 임시 분석 수행
c.analysis = codeBitmap(c.Code)
return  c.analysis.codeSegment(udest)
}
```

코드 예시

기본 EVM 사용법

```
// EVM 인스턴스 생성
blockCtx := vm.BlockContext{
CanTransfer: core.CanTransfer,
Transfer: core.Transfer,
GetHash: func(uint64) common.Hash { return common.Hash{} },
Coinbase: common.Address{},
BlockNumber: big.NewInt(1),
Time: uint64(time.Now().Unix()),
Difficulty: big.NewInt(1),
GasLimit: 1000000,
}

txCtx := vm.TxContext{
Origin: common.HexToAddress("0x1234"),
GasPrice: big.NewInt(1),
}

evm := vm.NewEVM(blockCtx, statedb, chainConfig, vm.Config{})
evm.SetTxContext(txCtx)
```

```go
// 컨트랙트 호출
caller := common.HexToAddress("0x1234")
addr := common.HexToAddress("0x5678")
input := []byte("test")
gas := uint64(100000)
value := uint256.NewInt(0)

ret, leftOverGas, err := evm.Call(caller, addr, input, gas, value)
if  err != nil {
log.Printf("EVM call failed: %v", err)
} else {
log.Printf("Call result: %x, gas used: %d", ret, gas-leftOverGas)
}
```

컨트랙트 배포

```go
// 컨트랙트 배포
bytecode := common.Hex2Bytes("6080604052348015610010576000080fd5b50...")
deployGas := uint64(500000)
deployValue := uint256.NewInt(0)

ret, contractAddr, leftOverGas, err := evm.Create(caller, bytecode, deployGas, d
```

```
if  err != nil {
log.Printf("Contract deployment failed: %v", err)
} else {
log.Printf("Contract deployed at: %s, gas used: %d", contractAddr.Hex(), deployG
}
```

최적화 및 성능

점프 목적지 캐싱

EVM은 컨트랙트의 점프 목적지를 분석하고 캐싱하여 성능을 향상시킵니다:

```
type JumpDestCache interface {
Load(hash common.Hash) (BitVec, bool)
Store(hash common.Hash, analysis BitVec)
}
```

메모리 풀링

메모리와 스택은 풀링을 통해 할당 비용을 줄입니다:

```
var  memoryPool = sync.Pool{
New: func() any {
return &Memory{}
},
```

```
}


var  stackPool = sync.Pool{
New: func() interface{} {
return &Stack{data: make([]uint256.Int, 0, 16)}
},
}
```

프리컴파일된 컨트랙트

자주 사용되는 암호화 함수들은 프리컴파일된 컨트랙트로 최적화됩니다:

```
type PrecompiledContract interface {
RequiredGas(input []byte) uint64
Run(input []byte) ([]byte, error)
}


func (evm *EVM) precompile(addr common.Address) (PrecompiledContract, bool) {
p, ok := evm.precompiles[addr]
return  p, ok
}
```

디버깅 및 모니터링

트레이싱 지원

EVM은 실행 과정을 추적할 수 있는 트레이싱 기능을 제공합니다:

```
type Config struct {
Tracer *tracing.Hooks
NoBaseFee  bool
EnablePreimageRecording  bool
ExtraEips []int
}


func (evm *EVM) captureBegin(depth  int, typ OpCode, from common.Address, to com
tracer := evm.Config.Tracer
if  tracer.OnEnter != nil {
tracer.OnEnter(depth, byte(typ), from, to, input, startGas, value)
}
}
```

에러 처리

EVM은 다양한 실행 에러를 정의하고 처리합니다:

```
var (
ErrOutOfGas = errors.New("out of gas")
ErrCodeStoreOutOfGas = errors.New("contract creation code storage out of gas")
```

```
ErrDepth = errors.New("max call depth exceeded")

ErrInsufficientBalance = errors.New("insufficient balance for transfer")

ErrContractAddressCollision = errors.New("contract address collision")

ErrExecutionReverted = errors.New("execution reverted")

)
```

관련 컴포넌트

상위 컴포넌트와의 연결

- **블록체인 코어 (02)**: `core.StateProcessor` 에서 EVM을 사용하여 트랜잭션 실행
- **트랜잭션 풀 (06)**: 트랜잭션 검증 시 EVM을 통한 시뮬레이션 실행
- **합의 엔진 (04)**: 블록 검증 과정에서 트랜잭션 실행 결과 확인

하위 컴포넌트와의 연결

- **스토리지 시스템 (07)**: `StateDB` 인터페이스를 통한 상태 데이터 접근
- **RPC API (08)**: `eth_call` , `eth_estimateGas` 등의 API에서 EVM 사용

EVM 실행 시스템은 이더리움의 핵심 기능인 스마트 컨트랙트 실행을 담당하며, 가스 기반의 안전한 실행 환경을 제공합니다. 스택 기반 아키텍처와 옵코드 시스템을 통해 결정론적이고 검증 가능한 계산을 수행합니다.